

PostgreSQL Backups: WAL-G

WAL-G Overview and its ancestor, WAL-E

Here the link to the video of the runbook simulation.

WAL-E was designed by Heroku to solve their PostgreSQL backup issues. It is a Python-based application that is invoked by the PostgreSQL process via the ‘archive_command’ as part of PostgreSQLs continuous archiving setup.

It works by taking Write-Ahead Logging files, compressing them, and then archiving them off to a storage target in near realtime. On a nightly schedule, WAL-E also pushes a full backup to the storage target, referred to as a ‘base backup’. A restore then is a combination of a ‘base backup’ and all of the WAL transaction files since the backup to recover the database to a given point in time.

WAL-G is the successor of WAL-E with a number of key differences. WAL-G uses LZ4, LZMA, or Brotli compression, multiple processors, and non-exclusive base backups for Postgres. It is backward compatible with WAL-E: it is possible to restore from a WAL-E backup using WAL-G, but not vice versa.

Currently (September 2021), the main backup tool for GitLab.com is WAL-G, we fully migrated to using it from WAL-E.

Very Quick Intro: 5 Main Commands

Backups consists of two parts:

- periodical (daily) “full” backups (a.k.a “base backups” executed by the `wal-g backup-push` command), and
- constant shipping of completed WAL files to GCS to enable Point-in-time recovery (PITR) (executed by the `wal-g wal-push` command - configured as postgresql `archive_command`).

To restore to a given point of time or to the latest available point, a full backup and a sequence of WALs are needed. To reduce the size of such a sequence, full backups can be performed more frequently. On the other hand, creation of full backups is a very IO-intensive operation, so it doesn’t make sense to do it very often. Currently, for the GitLab.com database, full backups are created daily.

WAL-G has 5 main commands:

Command	Purpose	How it is executed	Details
<code>backup-list</code>	Get the list of full backups currently stored in the archive	Manually	It is helpful to see if there are “gaps” (missing full backups). Also, based on displayed LSNs, we can calculate the amount of WALs generated per day. Notice, there is no “wal-list” - this command would print too much information so it would be hard to use it, so neither WAL-E nor WAL-G implement it. The backups in this list are ordered by the timestamp they were last modified on the filesystem. So if a backup gets moved to nearline storage after a certain time, it is wrongly shown as the newest! With <code>backup-list --detail --pretty</code> we get start and finish times and the backups are ordered correctly.

Command	Purpose	How it is executed	Details
<code>backup-push</code>	Create a full backup: archive PostgreSQL data directory fully	Manually or automatically	Daily execution is configured in a cron record (see <code>crontab -l</code> under <code>gitlab-psql</code>, it has an entry running the script <code>/opt/wal-g/bin/backup.sh</code>, writing logs to <code>/var/log/wal-g/wal-g_backup_push.log[.1]</code>) At the moment, it is executed daily (at 00:00 UTC) on one of the secondaries (chosen using a lock in Consul), using WAL-G. This operation is very IO-intensive, the expected speed: ~0.5-1 TiB/h for WAL-G, 1-2 TiB/h for WAL-G.

Command	Purpose	How it is executed	Details
<code>wal-push</code>	Archive WALs. Each WAL is 16 MiB by default	Automatically (<code>archive_command</code>)	This command is usually used in <code>archive_command</code> (PostgreSQL configuration parameter) and automatically executed by PostgreSQL on the primary node. As of June 2020, ~1.5-2 TiB of WAL files is archived each working day (less on holidays and weekends).
<code>backup-fetch</code>	Restore PostgreSQL data directory from a full backup	Manually	Is it to executed manually a fresh restore from backups is needed. Also used in “gitlab-restore” for daily verification of backups (see https://ops.gitlab.net/gitlab-com/gl-infra/gitlab-restore/postgres-prd/-/blob/master/bootstrap.sh).

Command	Purpose	How it is executed	Details
<code>wal-fetch</code>	Get a WAL from the archive	Automatically (<code>restore_command</code> ; not used on <code>patroni-XX</code> nodes)	It is to be used in (<code>restore_command</code> (see <code>recovery.conf</code> in the case of PostgreSQL 11 or older, and <code>postgresql.conf</code> for PostgreSQL 12+). Postgres automatically uses it to fetch and replay a stream of WALs on replicas. As of June 2020, <code>restore_command</code> is NOT configured on production and staging instances - we use only streaming replication there. However, in the future, it may change. Two “special” replicas, “archive” and “delayed”, do not use streaming replication - instead, they rely on fetching WALs from the archive, therefore, they have <code>wal-fetch</code> present in <code>restore_command</code> .

Backing Our Data Up

Where is Our Data Going

The data gets pushed using WAL-G to Google Cloud Storage into a bucket. All servers of an environment (like `gprd`) push their WAL to the same bucket location. This is because, in the event of a failover, all the servers should have the same backup location to streamline both backups and restores. With WAL-G, we use one of replica daily to create a full backup (`wal-g backup-push`) but we archive WALs (`wal-g wal-push` from the primary for better PRO. Some replicas retrieve WALs from the bucket for archive recovery (per configured `restore_command` in Postgres).

All GCS buckets are multi-regional storage in the US location.

environment	bucket
gprd	https://console.cloud.google.com/storage/browser/gitlab-gprd-postgres-backup
gstg	https://console.cloud.google.com/storage/browser/gitlab-gstg-postgres-backup
prdsdb	https://console.cloud.google.com/storage/browser/gitlab-prdsdb-postgres-backup
stgsdb	https://console.cloud.google.com/storage/browser/gitlab-stgsdb-postgres-backup
sentry	https://s3.console.aws.amazon.com/s3/buckets/gitlab-secondarydb-backups?region=us-east-1&prefix=sentry/&showversions=false

Interval and Retention

We currently take a basebackup each day at 0 am UTC and continuously stream WAL data to GCS. As of September 2021, the daily backup process performed by WAL-G takes ~9 hours with Postgres cluster size ~14 TiB.

Backups are kept for 7 days, moved to the Coldline storage class after 7 days, and deleted after 90 days by a lifecycle rule on GCS (check the production documentation for changes).

How Does it Get There?

Production

Daily basebackup

- `gprd/gstg`: `gitlab-walg` cookbook installs WAL-G and a cron job to run every day.

```
$ ssh patroni-v12-01-db-gprd.c.gitlab-production.internal -- 'sudo -u gitlab-psql crontab -e'
# Chef Name: full wal-g backup
0 0 * * * /opt/wal-g/bin/backup.sh >> /var/log/wal-g/wal-g_backup_push.log 2>&1
```

- `prds/sub/stgsub`: `walg` ansible role installs WAL-G and a cron job to run every day.

```
$ ssh customers-01-inf-prds.c.gitlab-subscriptions-prod.internal -- 'sudo -u postgres crontab -e'
#Ansible: Database backup: wal-g backup-push
0 0 * * * /usr/local/bin/envdir /etc/wal-g.d/env /usr/local/bin/wal-g backup-push /var/log/wal-g/wal-g_backup_push.log 2>&1
```

Archiving WALs WAL files are sent via PostgreSQL's `archive_command` parameter, which looks something like the following:

```
archive_command = /opt/wal-g/bin/archive-walg.sh %p
```

How Do I Verify This?

You can always check the GCS storage bucket and see the contents of the basebackup and WAL archive folders.

Alternatively, you can check the results of continuous WAL archiving and daily full backup creation manually on servers:

How to check WAL archiving See `/var/log/wal-g/wal-g.log` on primary:

```
$ sudo tail /var/log/wal-g/wal-g.log
INFO: 2021/09/02 15:57:42.138682 FILE PATH: 000000050005E57500000016.br
INFO: 2021/09/02 15:57:42.156047 FILE PATH: 000000050005E57500000015.br
INFO: 2021/09/02 15:57:42.458779 FILE PATH: 000000050005E57500000017.br
INFO: 2021/09/02 15:57:42.662657 FILE PATH: 000000050005E57500000018.br
INFO: 2021/09/02 15:57:43.858475 FILE PATH: 000000050005E57500000019.br
INFO: 2021/09/02 15:57:43.863675 FILE PATH: 000000050005E5750000001B.br
INFO: 2021/09/02 15:57:43.871022 FILE PATH: 000000050005E5750000001C.br
INFO: 2021/09/02 15:57:44.124049 FILE PATH: 000000050005E5750000001A.br
INFO: 2021/09/02 15:57:44.189580 FILE PATH: 000000050005E5750000001D.br
INFO: 2021/09/02 15:57:44.482529 FILE PATH: 000000050005E5750000001E.br
```

See also: How to check if WAL-G backups are running.

How to check the full backups (basebackups) creation Check `/var/log/wal-g/wal-g_backup_push.log` / `/var/log/wal-g/wal-g_backup_push.log.1` on replicas:

```
$ sudo tail -n30 /var/log/wal-g/wal-g_backup_push.log.1
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.653809 Starting part 14244 ...
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.653851 /global/pg_control
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.662161 Finished writing part 14244
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.663456 Calling pg_stop_backup()
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.665868 Starting part 14245 ...
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.665917 backup_label
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.665924 tablespace_map
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:54.669692 Finished writing part 14245
<13>Sep 2 00:00:01 backup.sh: INFO: 2021/09/02 08:20:56.337685 Wrote backup with name base
<13>Sep 2 00:00:01 backup.sh: end backup pgbackup_pg12-patroni-cluster_20210902.
<13>Sep 2 00:00:01 backup.sh: * Trying 10.219.1.3...
<13>Sep 2 00:00:01 backup.sh: * Connected to blackbox-01-inf-gprd.c.gitlab-production.internal:9090
<13>Sep 2 00:00:01 backup.sh: > POST /metrics/job/walg-basebackup/shard/default/tier/db/typ
<13>Sep 2 00:00:01 backup.sh: > Host: blackbox-01-inf-gprd.c.gitlab-production.internal:9090
<13>Sep 2 00:00:01 backup.sh: > User-Agent: curl/7.47.0
<13>Sep 2 00:00:01 backup.sh: > Accept: */*
<13>Sep 2 00:00:01 backup.sh: > Content-Length: 198
<13>Sep 2 00:00:01 backup.sh: > Content-Type: application/x-www-form-urlencoded
<13>Sep 2 00:00:01 backup.sh: >
<13>Sep 2 00:00:01 backup.sh: } [198 bytes data]
<13>Sep 2 00:00:01 backup.sh: * upload completely sent off: 198 out of 198 bytes
<13>Sep 2 00:00:01 backup.sh: < HTTP/1.1 200 OK
<13>Sep 2 00:00:01 backup.sh: < Date: Thu, 02 Sep 2021 08:20:56 GMT
<13>Sep 2 00:00:01 backup.sh: < Content-Length: 0
<13>Sep 2 00:00:01 backup.sh: <
<13>Sep 2 00:00:01 backup.sh: * Connection #0 to host blackbox-01-inf-gprd.c.gitlab-production.internal:9090
<13>Sep 2 00:00:01 backup.sh: HTTP/1.1 200 OK
<13>Sep 2 00:00:01 backup.sh: Date: Thu, 02 Sep 2021 08:20:56 GMT
<13>Sep 2 00:00:01 backup.sh: Content-Length: 0
<13>Sep 2 00:00:01 backup.sh:
```

Checklist of reconfiguration / WAL-G upgrades After upgrades or re-configuration check:

1. Check that WAL-G binary is working and shows the expected version:

```
cd /tmp
sudo -u gitlab-psql /usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g --version
```

2. Check the logs (as described above):

- WAL archiving (check this on the primary)

```
sudo tail /var/log/wal-g/wal-g.log
```

- full backup creation (check all replicas or only that one where you know that backup-push is happening)

```
sudo tail /var/log/wal-g/wal-g_backup_push.log.1
```

3. Once a new full backup is created, check the list of full backups available:

```
sudo -u gitlab-psql /usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-list
```

(Important note: the output shows modification timestamps. In WAL-G versions prior 1.1, the list was ordered by modification time; since 1.1 it's ordered by creation time, which is important for our case, when we automatically move older backups to the Nearline storage class)

4. After 1-2 days, check that verification jobs ("gitlab-restore" project) are not failing.

Restoring Data

Oh Sh*t, I Need to Get It BACK

Before we start, take a deep breath and don't panic.

TODO: update to WAL-G

Production Consider using the delayed replica to speed up PITR. The full database backup restore is also automated in a CI pipeline, which may be helpful depending on the type of disaster. To restore from WAL-E backups, either WAL-G or WAL-E can be used. In "gitlab-restore", the default is WAL-G, as it gives 3-4 times better restoration speed than WAL-E. Use `WAL_E_OR_WAL_G` CI variable to switch to WAL-E if needed (just set this variable to `wal-e`). For the "basebackup" phase of the restore process, on an `n1-standard-16` instance, the expected speed of filling the PGDATA directory is 0.5-1 TiB per hour for WAL-E and 1.5-2 TiB per hour for WAL-G.

Below we describe the restore process step by step for the case of WAL-E (old procedure). For WAL-G, it is very similar, with a couple of nuances. For details, please see <https://ops.gitlab.net/gitlab-com/gl-infra/gitlab-restore/postgres-gprd/blob/master/bootstrap.sh>.

In order to restore, the following steps should be performed. It is assumed that you have already set up the new server, and that server is configured with our current chef configuration.

1. In the Chef role assigned to the node, add `recipe[gitlab-server::postgresql-standby]` to the runlist, apply the following attributes then converge Chef:

```
{
  "gitlab-server": {
    "postgresql-standby": {
      "postgres-conf-auto-rules": {
        "restore_command": "/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g wal-f
        "recovery_target_timeline": "latest"
```

```

    }
  }
}

```

2. Log in to the `gitlab-psql` user (`su - gitlab-psql`)
3. Restore the base backup (run in a screen on tmux session and be ready to wait several hours):

```

/usr/bin/envdir /etc/wal-e.d/env /opt/wal-e/bin/wal-e backup-list
PGHOST=/var/opt/gitlab/postgresql/ PATH=/opt/gitlab/embedded/bin:/opt/gitlab/embedded/s
/usr/bin/envdir /etc/wal-e.d/env /opt/wal-e/bin/wal-e backup-fetch /var/opt/gitlab/po

```

To restore latest backup you can use the following:

```

/usr/bin/envdir /etc/wal-e.d/env /opt/wal-e/bin/wal-e backup-list
PGHOST=/var/opt/gitlab/postgresql/ PATH=/opt/gitlab/embedded/bin:/opt/gitlab/embedded/s
/etc/wal-e.d/env /opt/wal-e/bin/wal-e backup-fetch /var/opt/gitlab/postgresql/data LA

```

4. This command will output nothing if it is successful.
5. Optional: In case the database should only be recovered to a certain point-in-time, add recovery target settings to `postgresql.auto.conf`.
6. Start PostgreSQL. This will begin the archive recovery. You can watch the progress in the postgres log.
7. IMPORTANT:
 - WAL-G won't stores the PostgreSQL configuration (`postgresql.conf`, `postgresql.auto.conf`). You need to take care about configuration separately.
 - As of June 2020, `restore_command` is not used on `patroni-XX` nodes. Therefore, if your restoration is happening as a part of DR, you need to consider removing `restore_command` in the very end.
 - If this is your new master (again, if it is DR actions), then you need to promote it (using `/var/opt/gitlab/postgresql/trigger` or `pg_ctl promote`), and adjust configuration (`archive_command`, `restore_command`).

Troubleshooting

NB: the document below is slightly outdated, refering WAL-E, but in general, WAL-E instructions are relevalt to WAL-G TODO: update it

How to Check if WAL-G Backups are Running

Daily backups (“base backups” or “`wal-g_backup_push`”) WAL-G `/opt/wal-g/bin/backup.sh` script is running on all machines in the patroni

cluster. However, the “base backups” actually runs only in the **first Replica that acquires a Consul lock** for the job execution.

You can check WAL-G `backup-push` in several ways:

1. Metric (injected via prometheus push-gateway from the backup script):
gitlab_com:last_walg_successful_basebackup_age_in_hours
2. using Kibana (not valid while wal-g logs are not yet shipped to Elastic Search):
 - log.gprd.gitlab.net
 - index: pubsub-system-inf-gprd
 - document field: json.ident with value wal_g.worker.upload
1. by logging directly into the VM:
 - ssh to the patroni master
 - logs are located in `/var/log/wal-g/wal-g_backup_push.log`, the file is under rotation, so check also `/var/log/wal-g/wal-g_backup_push.log.1`, etc

As WAL-G `backup-push` only executes in one of the replicas, you should observe the following logs:

`backup-push` on the Primary: example of log entries on the primary

```
<DATE> backup.sh: I'm the primary. Will not run backup.
```

`backup-push` on the Replica that executed the backup (only the first replica that starts the backup should execute it):

```
<DATE> backup.sh: start backup <LABEL> ...
...
<DATE> backup.sh: INFO: <DATE> Selecting the latest backup as the base for the current delta
<DATE> backup.sh: INFO: <DATE> Calling pg_start_backup()
<DATE> backup.sh: INFO: <DATE> Starting a new tar bundle
<DATE> backup.sh: INFO: <DATE> Walking ...
<DATE> backup.sh: INFO: <DATE> Starting part 1 ...
...
<DATE> backup.sh: INFO: <DATE> /global/pg_control
<DATE> backup.sh: INFO: <DATE> Finished writing part ???
<DATE> backup.sh: INFO: <DATE> Calling pg_stop_backup()
<DATE> backup.sh: INFO: <DATE> Starting part ??? ...
<DATE> backup.sh: INFO: <DATE> backup_label
<DATE> backup.sh: INFO: <DATE> tablespace_map
<DATE> backup.sh: INFO: <DATE> Finished writing part ???
<DATE> backup.sh: INFO: <DATE> Wrote backup with name base_???
<DATE> backup.sh: end backup <LABEL>.
```

`backup-push` on the Replica that did not execute the backup:

```
<DATE> backup.sh: start backup ??? ...
```

```
<DATE> backup.sh: Shutdown triggered or timeout during lock acquisition
<DATE> backup.sh: Could not acquire wal-g-basebackup-patroni lock. Will not run backup.
```

Continuous Shipping of WAL Files (“wal-push”) As described in the previous chapter, the base backups are taken in one of the replicas, however, the WAL (transaction log files) are continuously generated and shipped into the cloud storage from the Primary node (only node accepting writes).

In order to find out which machine is the primary, go to the relevant Grafana dashboard or execute `sudo gitlab-patronictl list` on one of the nodes.

WAL-G **wal-push** works by uploading WAL files to a GCS bucket whenever a wal file is completed (which happens every few seconds in gprd). The **wal-g wal-push** command is configured as postgresql `archive_command` and run by a PostgreSQL background worker. The worker can run custom code, in our case, it’s running the WAL-G `/opt/wal-g/bin/wal-g wal-push` binary. The background worker is a process that is forked from the main postgres process. In case of WAL-G **wal-push** it lives only a few seconds.

Any relevant information will be logged into `/var/log/wal-g/wal-g.log` file, including pushed files and errors. > *See more at:* <https://gitlab.com/gitlab-com/runbooks/-/blob/master/docs/patroni/postgresql-backups-wale-walg.md#how-to-check-wal-archiving>

In the PostgreSQL logs we found just the errors of the `archive_command`, and you can check them with the following:

```
grep "wal-g" /var/log/gitlab/postgresql/postgresql.csv
```

For each failed attempt to archive a wal file, there would be an entry like the following:

```
"archive command failed with exit code 1","The failed archive command was: /opt/wal-g/bin/ar
```

Continuous Shipping of WAL Files (wal-push) is not working

Metric (via mtail): `gitlab_com:last_walg_backup_age_in_seconds` metric

Attention: If WAL shipping (`archive_command`) fails for some reason, WAL files will be kept on the server until the disk is running full! Check disk available space of Prod Patroni servers

WAL-G wal-push process stuck If you’re not seeing logs for successful **wal-push** uploads (e.g. nothing writes to the log file or there are only entries with `state=begin` but not with `state=complete`) then there’s something wrong with WAL-E.

Check the WAL-E’s wal-push upload process Run `ps` a few times (the upload process is short-lived so you might not catch it the first time), example output:

```
# ps aux | grep 'wal-push'
gitlab-+ 29632  0.0  0.0   4500   844 ?        S    12:16   0:00 sh -c /usr/bin/envdir /etc/
gitlab-+ 29633 35.0  0.0  124200 41488 ?        D    12:16   0:00 /opt/wal-e/bin/python /opt/
root      29638  0.0  0.0   12940   920 pts/0    S+   12:16   0:00 grep wal-push
```

If the timestamp on the WAL-E process is relatively long time in the past (e.g. 15 mins, 1h) then that's a hint that it's stuck at uploading files.

Check the state of the process with: `strace -p <pid>`. If the process is stuck, `strace` will show no activity.

Another indicator of a stuck process is the timestamp on the latest file uploaded to GCS, i.e. it will be close to the timestamp on the upload process. `gsutil` might take too long to list files in the bucket, so go to the web UI and start typing in the prefix of the filename last uploaded (don't type in the full name).

If everything points to the fact that WAL-E upload process is stuck, consider killing it. BE EXTREMELY CAREFUL! After killing the process it should be restarted automatically and the backups should resume immediately.

Decommissioned node alert To remove metrics from the pushgateway check how to delete metrics

Other If `wal-push` is not working, it will probably be something related with the network or GCS.

PostgreSQL is configured to archive with WAL-E upon some conditions, as specified via Chef:

```
gitlab_rb:
  postgresql:
    archive_command: /usr/bin/envdir /etc/wal-e.d/env /opt/wal-e/bin/wal-e
```

WAL-E wal-push is working (but I still got paged)

The problem might be `mtail`.

1. Check `mtail` is working with `sudo sv status mtail`
2. If `mtail` is up, check `/var/log/mtail` for errors under `/var/log/mtail.ERROR`.
3. You might want to restart `mtail` if it's stuck with `sudo sv restart mtail`.

Regular basebackup has failed

1. Check the logs in `/var/log/wal-g/wal-g_backup_push.log` (and possibly rotated files).
2. If the error looks to be a transient GCS error, you can retry the backup in a `tmux` with:

```
sudo su - gitlab-psql
/opt/wal-g/bin/backup.sh >> /var/log/wal-g/wal-g_backup_push.log 2>&1
```

Transient object store errors should be retried once this PR lands in a wal-g release that we consume: <https://github.com/wal-g/wal-g/pull/833>.

Database Backups Restore Testing

Backups restore testing is fully automated with GitLab CI pipelines, see <https://ops.gitlab.net/gitlab-com/gl-infra/gitlab-restore>.

1. **postgres-gprd**: Backups of production GitLab.com backup are tested twice per day:
 1. Slightly after the time when **backup-push** is expected to be finished on the primary (at 11:30 a.m. UTC as of June 2020). This verifies the fresh full backup and small addition of WALs.
 2. Right after **backup-push** is invoked on the primary (at 00:05 a.m.). This allows to ensure that not only full backups are in a good state, but also WAL stream, all the WALs in the archive are OK, without gaps.
2. **postgres-prdsub**: Backups of production customers.GitLab.com are tested once per day
3. **Dev**: Backups of dev.gitlab.org are tested once per day
4. **Registry**: Backups of the registry database are tested once per day

When a backup is verified via the CI job, the job will send a notification to Dead Man's Snitch. If the job fails to check in after a specified number of times, Dead Man's Snitch will send a page.

Troubleshooting: What to Do if Backup Restore Verification Fails

In the case of failing Postgres backup verification jobs, use the following to troubleshoot:

1. In **gitlab-restore** find the pipeline that is subject to investigation and remember its ID.
2. First, look inside the pipeline's jobs output. Sometimes the instance even hasn't been provisioned – quite often due to hitting some quotas (such as number of vCPUs or IP addresses in “gitlab-restore” project). In this case, either clean up instances that are not needed anymore or increase the quotas in GCP.
3. In “gitlab-restore” project at GCP console, find an instance with the pipeline ID in instance name. SSH to it and check:
 - Disk space (**df -hT**). If we hit the disk space limit, it is time to increase the disk size again – usually, it's done in the source code of the “gitlab-restore” project, but it is also possible to configure CI/CD schedules to override it.

- Recent logs (`sudo journalctl -f`, `sudo journalctl --since yesterday | less`). There might be some insights related to, say, WAL-E/WAL-G failures.
 - Postgres replica is working (`sudo gitlab-psql`). If you cannot connect, then either Postgres is not installed properly, or it hasn't reached the point when PGDATA can be considered consistent. If the replaying of WALs is still happening (see the logs), then it is worth waiting some time. Otherwise, the logs should be carefully investigated.
4. Finally, if none of above items revealed any issues, try performing `backup-fetch` manually. For that:
 1. Run a new CI/CD pipeline in "gitlab-restore", using the CI variable values taking them from "Schedules" section (there is "Reveal" button there), and adding `NO_CLEANUP = 1` to preserve the instance.
 2. SSH to the instance after a few minutes, when it's up and running.
 3. Before proceeding, use WAL-E's (WAL-G's) `backup-list` to see the available backups. One of possible reasons of failure is lack of some daily basebackup. In such a case, you need to so to Postgres master node and analyze WAL-E (WAL-G) log (check `sudo -u postgres crontab -l`, it should show how daily basebackups are triggered and where the logs are located). If the list of backups looks right, continue troubleshooting.
 4. Wait until the issue repeats and backup verification fails (assuming it's permanent – if not, we only can analyze the logs of the previous runs).
 5. Manually follow the steps from <https://ops.gitlab.net/gitlab-com/gl-infra/gitlab-restore/postgres-gprd/blob/master/bootstrap.sh>, starting from erasing PGDATA directory and proceed to WAL-E's (WAL-G's) `backup-fetch` step which normally takes a few hours.
 6. Once `backup-fetch` is finished, you should have a Postgres "archive replica" – a Postgres instance that constantly pulls new WAL data from WAL-E (WAL-G) archive. Check it with `sudo gitlab-psql`. Note, that it is normal if you cannot connect during some period of time and see **FATAL: the database system is starting up** error: until recovery mode is reached a consistent point, Postgres performs REDO and doesn't allow connections. It may take some time (minutes, dozens of minutes), after which you should be able to connect and observe how the database state is constantly changing due to receiving (via `wal-fetch`) and replaying new WALs. To see that, use either `select pg_last_xact_replay_timestamp()` or `select now(), created_at, now() - created_at from issues order by id desc limit 1`.
 7. Troubleshoot any failures in place, checking the logs, free disk space and so on.
 8. Finally, once troubleshooting is done, do not forget to destroy the instance manually, it won't get destroyed automatically because of

`NO_CLEANUP = 1` we have used!

Further Read

- Continuous Archiving and Point-in-Time Recovery (PITR) (PostgreSQL official documentation)
- Backup Control Functions (PostgreSQL official documentation)
- WAL Internals (PostgreSQL official documentation)
- Write Ahead Logging — WAL (The Internals of PostgreSQL)
- Understanding WAL nomenclature (Euler Taveira)
- What does `pg_start_backup()` do? (2nd Quadrant)